Arquitecturas

Estilos y coordinación

Redes de Comunicaciones y Cómputo Distribuido

22 de Octubre de 2026

Arquitecturas: ¿de qué hablamos?

Arquitectura de software: organización lógica

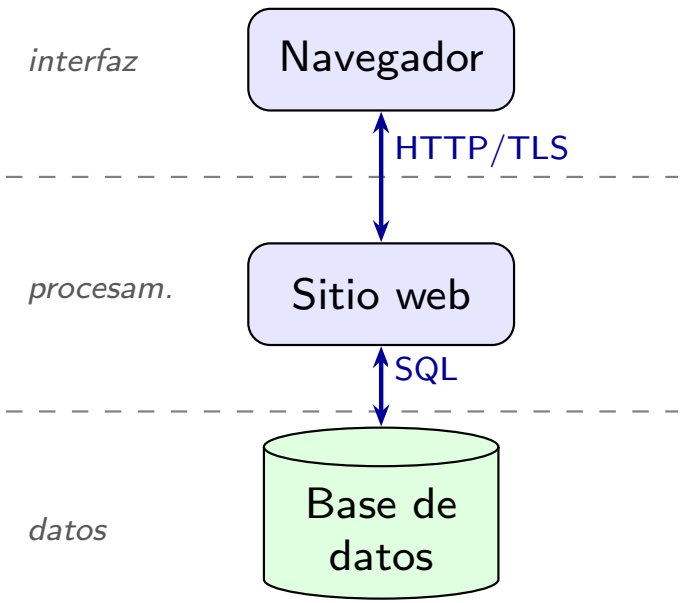
Qué componentes hay, cómo se conectan y cómo interactúan. **Hoy.**

Arquitectura de sistema: organización física

Dónde se instancian esos componentes: en qué máquina corre cada uno. **Próxima clase.**

Bibliografía: M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Capítulo 2: Architectures.

Convención para los diagramas

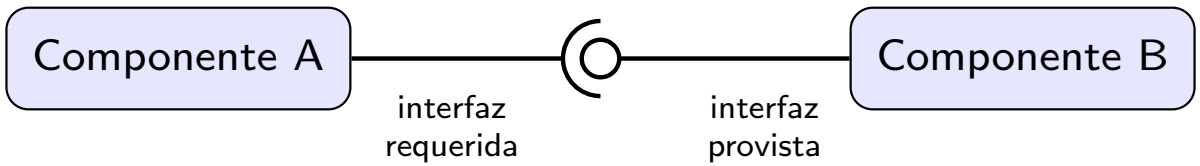


- **Caja** = componente.
- **Flecha** = conector, etiquetada con el mecanismo y/o el protocolo ("HTTP/TLS", "WebSocket", "cola").
- **Línea punteada** = límite entre niveles lógicos: interfaz / procesamiento / datos.
- Próxima clase: un **recuadro** alrededor de las cajas marca la máquina (tier físico).

Componentes y conectores

Componente

Unidad modular con interfaces **provistas** y **requeridas** bien definidas, que es **reemplazable** dentro de su entorno.



- Se puede reemplazar, incluso con el sistema andando, **sólo si su interfaz no cambia**.

Conector

Mecanismo que media la comunicación, la coordinación o la cooperación entre componentes: llamadas a procedimientos (remotos), pasaje de mensajes, *streams* de datos.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., §2.1.

Estilos arquitectónicos

Estilo arquitectónico

Se define en términos de:

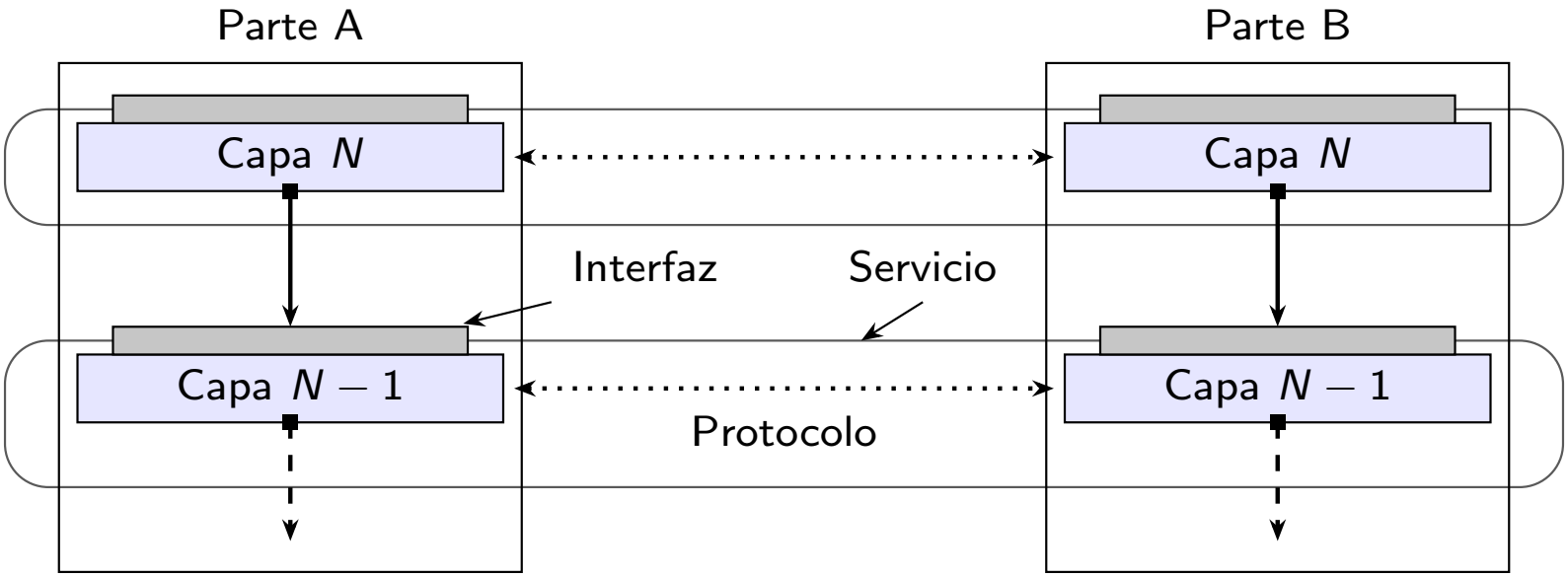
- los **componentes**,
- cómo se **conectan** entre sí,
- los **datos** que intercambian,
- y cómo se **configuran** juntos para formar el sistema.

Los más importantes para sistemas distribuidos:

1. En capas (*layered*)
2. Orientados a servicios (*service-oriented*)
3. Publish-subscribe

En los sistemas reales se combinan. Dividir en capas lógicas es tan universal que aparece junto a casi cualquier otro estilo.

Servicio, interfaz y protocolo



- **Servicio**: lo que ofrece una capa (p.ej. transporte confiable orientado a conexión).
- **Interfaz**: las funciones con que se accede al servicio; oculta su implementación.
- **Protocolo**: las reglas que siguen las partes para intercambiar información (p.ej. TCP).

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Fig. 2.2.

Ejemplo: dos partes que se comunican

Servidor

```
from socket import *
s = socket(AF_INET, SOCK_STREAM)
(conn, addr) = s.accept()
while True:
  data = conn.recv(1024)
  if not data: break
  msg = data.decode()+"*"
  conn.send(msg.encode())
conn.close()
```

Cliente

```
from socket import *
s = socket(AF_INET, SOCK_STREAM)
s.connect((HOST, PORT))
msg = "Hello World"
s.send(msg.encode())
data = s.recv(1024)
print(data.decode())
s.close()
```

¿Qué es la interfaz, cuál es el servicio y cuál el protocolo? ¿Hay algún otro protocolo?

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Nota 2.1 (código abreviado).

Ejemplo: dos partes que se comunican

Concepto	**En el código**
Interfaz	`socket()`, `accept()`, `connect()`, `send()`, `recv()`, `close()`. Las constantes `AF_INET` y `SOCK_STREAM` también son parte de la interfaz.

Ejemplo: dos partes que se comunican

Concepto	**En el código**
Interfaz	`socket()`, `accept()`, `connect()`, `send()`, `recv()`, `close()`. Las constantes `AF_INET` y `SOCK_STREAM` también son parte de la interfaz.
Servicio	Transporte confiable y orientado a conexión entre las dos partes.

Ejemplo: dos partes que se comunican

Concepto	**En el código**
Interfaz	`socket()`, `accept()`, `connect()`, `send()`, `recv()`, `close()`. Las constantes `AF_INET` y `SOCK_STREAM` también son parte de la interfaz.
Servicio	Transporte confiable y orientado a conexión entre las dos partes.
Protocolo	TCP. Cómo está implementado queda oculto a las aplicaciones.

Ejemplo: dos partes que se comunican

Concepto	**En el código**
Interfaz	`socket()`, `accept()`, `connect()`, `send()`, `recv()`, `close()`. Las constantes `AF_INET` y `SOCK_STREAM` también son parte de la interfaz.
Servicio	Transporte confiable y orientado a conexión entre las dos partes.
Protocolo	TCP. Cómo está implementado queda oculto a las aplicaciones.
Protocolo de aplicación	**Implícito**: el cliente manda un texto y el servidor lo devuelve con un "*" agregado (un servidor de eco).

Capas de una aplicación: tres niveles lógicos

Muchas aplicaciones distribuidas se arman con tres piezas:

Nivel de interfaz

Maneja la interacción con el usuario o con otra aplicación externa.

Nivel de procesamiento

El núcleo funcional de la aplicación.

Nivel de datos

Opera sobre una base de datos o un sistema de archivos.

Capas de una aplicación: tres niveles lógicos

Muchas aplicaciones distribuidas se arman con tres piezas:

Nivel de interfaz

Maneja la interacción con el usuario o con otra aplicación externa.

Nivel de procesamiento

El núcleo funcional de la aplicación.

Nivel de datos

Opera sobre una base de datos o un sistema de archivos.

¿Podrían correr los tres en la misma máquina?

Sí: son niveles **lógicos**, no máquinas. Dónde corre cada uno es otra decisión (la arquitectura de sistema).

Ejercicio 1.b como ejemplo

Ej 1

Diseñar los sistemas distribuidos asociados a los siguientes escenarios, desde el punto de vista de sus componentes y conectores. Describir los protocolos que intervienen y explicar los niveles de interfaz, procesamiento y datos.

b. Desde 10 navegadores web en 10 PCs se realizan 10 búsquedas de alquiler de propiedades en un sitio web. El sitio web almacena datos en un sistema distribuido en un datacenter.

Ejercicio 1.b como ejemplo

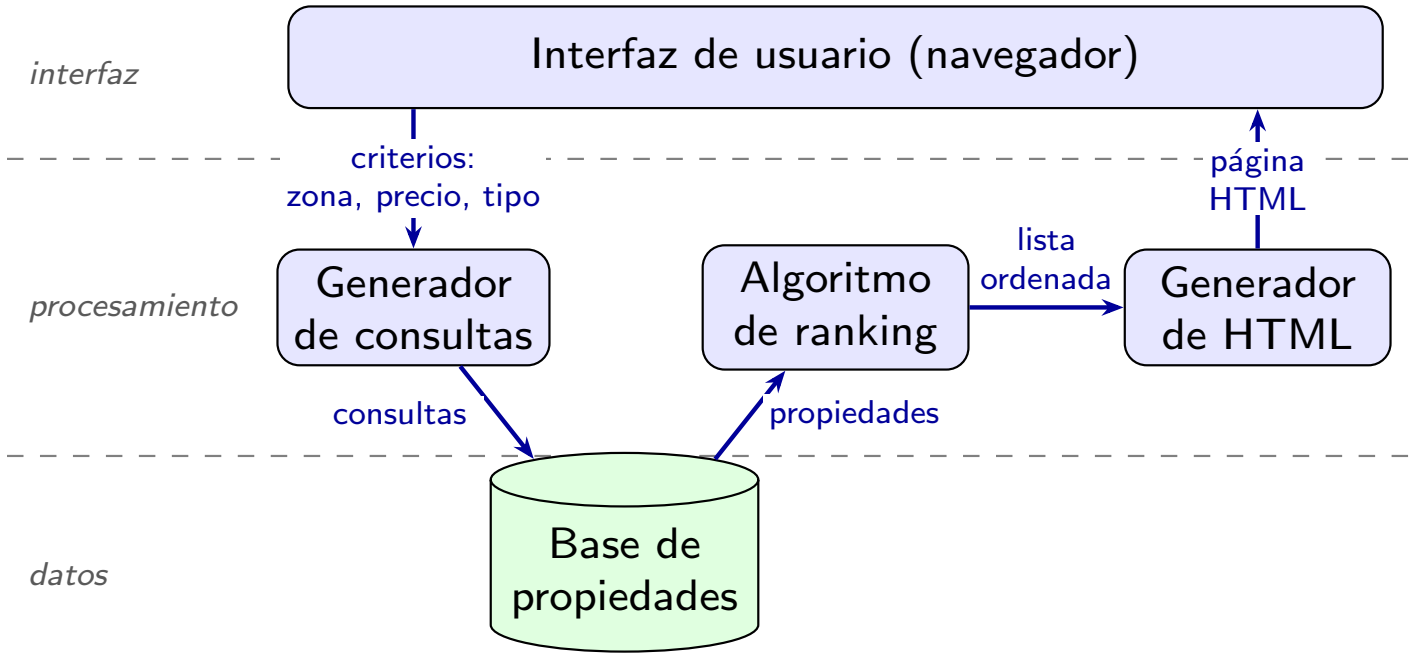
Ej 1

Diseñar los sistemas distribuidos asociados a los siguientes escenarios, desde el punto de vista de sus componentes y conectores. Describir los protocolos que intervienen y explicar los niveles de interfaz, procesamiento y datos.

b. Desde 10 navegadores web en 10 PCs se realizan 10 búsquedas de alquiler de propiedades en un sitio web. El sitio web almacena datos en un sistema distribuido en un datacenter.

- **Interfaz**: el navegador de cada PC (10 instancias del mismo componente).
- **Procesamiento**: el sitio web arma las consultas, ordena y genera el HTML.
- **Datos**: el almacenamiento distribuido del datacenter.
- **Conectores**: HTTP/TLS entre navegador y sitio (request-reply); SQL o RPC entre el sitio y el almacenamiento.

Ejemplo: un buscador de propiedades



El nivel de procesamiento sólo transforma los criterios en consultas, ordena las respuestas por relevancia y arma el HTML.

Las 10 búsquedas son independientes: si el sitio no guarda estado entre pedidos, se puede replicar y repartir la carga.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Fig. 2.4.

Problema de las capas

Dependencias fuertes entre capas. Caso extremo: retiraron de NPM un componente que rellenaba strings con ceros y se rompieron miles de programas.

Alternativa: entidades **independientes** que encapsulan un servicio y corren como procesos separados (objetos, servicios, microservicios).

Estilo basado en objetos

- Componente = objeto: **estado** + **métodos**.
- Conector = llamada a procedimiento, que puede ser remota.
- La interfaz oculta la implementación: el objeto es reemplazable por otro con la misma interfaz.

Microservicios y composición de servicios

Microservicios

- Cada uno corre como un proceso de red separado y es un servicio independiente: la clave es la **modularización**.
- No hay consenso sobre qué tamaño debe tener un microservicio.

Composición de servicios de grano grueso

- Los servicios pueden pertenecer a distintas organizaciones.
- Suelen ser servicios relativamente grandes, incluso a veces implementados internamente mediante microservicios.

Microservicios y composición de servicios

Microservicios

- Cada uno corre como un proceso de red separado y es un servicio independiente: la clave es la **modularización**.
- No hay consenso sobre qué tamaño debe tener un microservicio.

Composición de servicios de grano grueso

- Los servicios pueden pertenecer a distintas organizaciones.
- Suelen ser servicios relativamente grandes, incluso a veces implementados internamente mediante microservicios.

Armar el sistema es **componer servicios**, cada uno con su propia interfaz: la integración no es trivial.

Arquitecturas basadas en recursos: REST

Ver el sistema como una gran colección de **recursos**, cada uno manejado por un componente. Cuatro características:

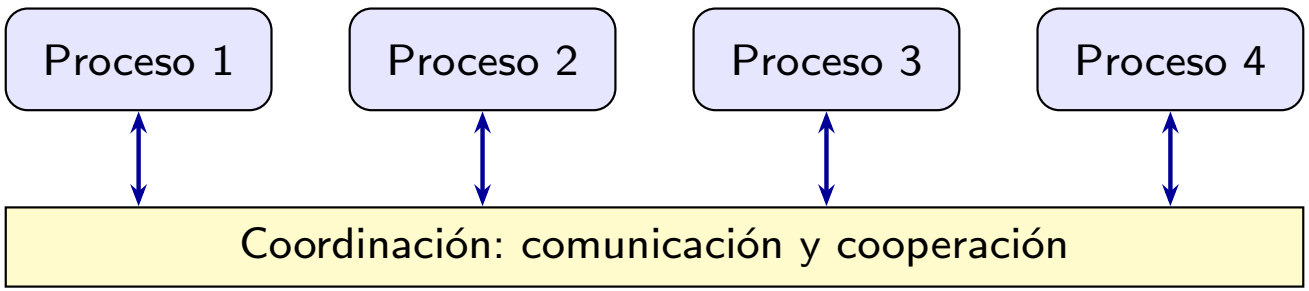
1. Los recursos se identifican con un único esquema de nombres (URI).
2. Todos los servicios ofrecen la misma interfaz, de a lo sumo cuatro operaciones.
3. Los mensajes que van y vienen de un servicio son autodescriptos.
4. Después de ejecutar una operación, el servicio se olvida de quién lo llamó: ejecución **stateless**.

Operación	**Descripción**
`PUT`	Modifica un recurso transfiriéndole un nuevo estado
`POST`	Crea un recurso nuevo
`GET`	Obtiene el estado de un recurso en alguna representación
`DELETE`	Borra un recurso

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Fig. 2.7.

Publish-subscribe: la idea

- A medida que los sistemas crecen, y los procesos entran y salen más fácilmente: conviene que las dependencias entre procesos sean lo más débiles posible.
- Para eso se separa el **procesamiento** de la **coordinación**.



- **Procesamiento**: cada proceso opera de forma autónoma.
- **Coordinación**: cómo se comunican y cooperan entre sí.
- Los procesos **publican** eventos y se **suscriben** a los que les interesan, sin conocerse entre sí.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., §2.1.3.

Coordinación: acoplamiento referencial y temporal

- **Acoplamiento referencial**: para comunicarse, un proceso tiene que conocer el nombre o identificador del otro.
- **Acoplamiento temporal**: los dos procesos tienen que estar andando al mismo tiempo.

	Temp. acoplado	**Temp. desacoplado**
Ref. acoplado	**Directa**	**Mailbox**
Ref. desacoplado	**Event-based**	**Shared data space**

¿Un ejemplo cotidiano para cada celda?

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Fig. 2.9.

Coordinación: acoplamiento referencial y temporal

- **Acoplamiento referencial**: para comunicarse, un proceso tiene que conocer el nombre o identificador del otro.
- **Acoplamiento temporal**: los dos procesos tienen que estar andando al mismo tiempo.

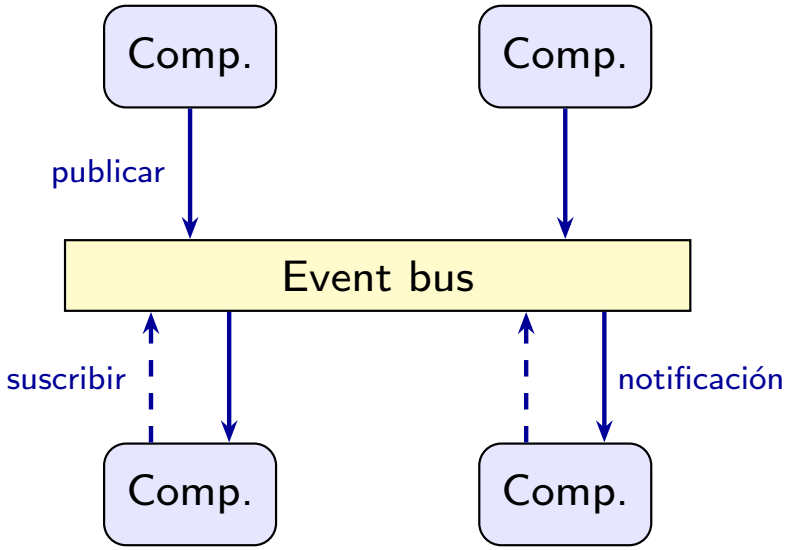
	Temp. acoplado	**Temp. desacoplado**
Ref. acoplado	**Directa** llamada telefónica	**Mailbox** mail, mensaje de voz
Ref. desacoplado	**Event-based** radio en vivo, alarma de incendio	**Shared data space** cartelera de avisos

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Fig. 2.9.

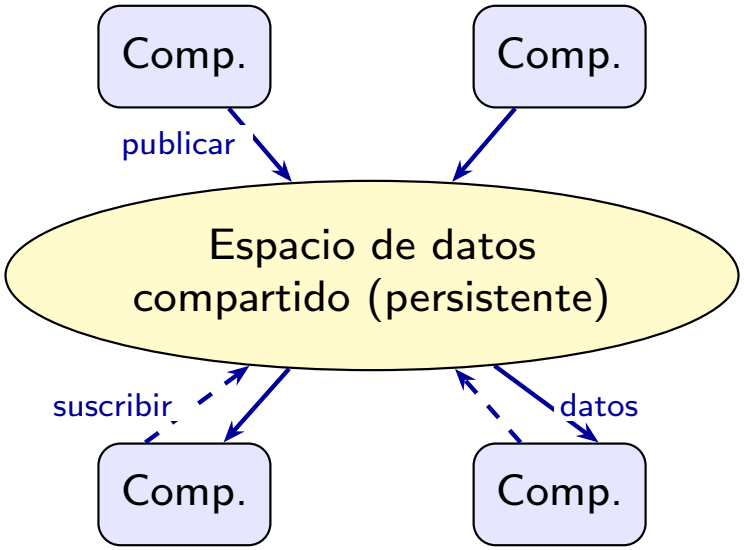
Las cuatro formas de coordinación

- **Directa**: los procesos se conocen y están activos a la vez.
- **Mailbox**: no hace falta que estén activos al mismo tiempo. Los mensajes se dejan en un buzón (posiblemente compartido) que hay que direccionar explícitamente.
- **Event-based**: los procesos no se conocen. Publican **notificaciones** de eventos y se **suscriben** a tipos de notificaciones. Pero el suscriptor tiene que estar andando cuando se publica.
- **Shared data space**: se comunican mediante **tuplas** (registros estructurados, como filas de una tabla) en un espacio compartido. Para leer se da un patrón, y vuelve cualquier tupla que coincida (búsqueda asociativa).

Arquitecturas publish-subscribe



(a) Event-based



(b) Shared data space

- Rasgo clave: los procesos **no tienen referencias explícitas** entre sí.
- El *event bus* abstrae el mecanismo que junta publicadores con suscriptores.
- Se suelen combinar: un proceso se suscribe dando un patrón y, cuando alguien inserta una tupla que coincide, se le notifica.

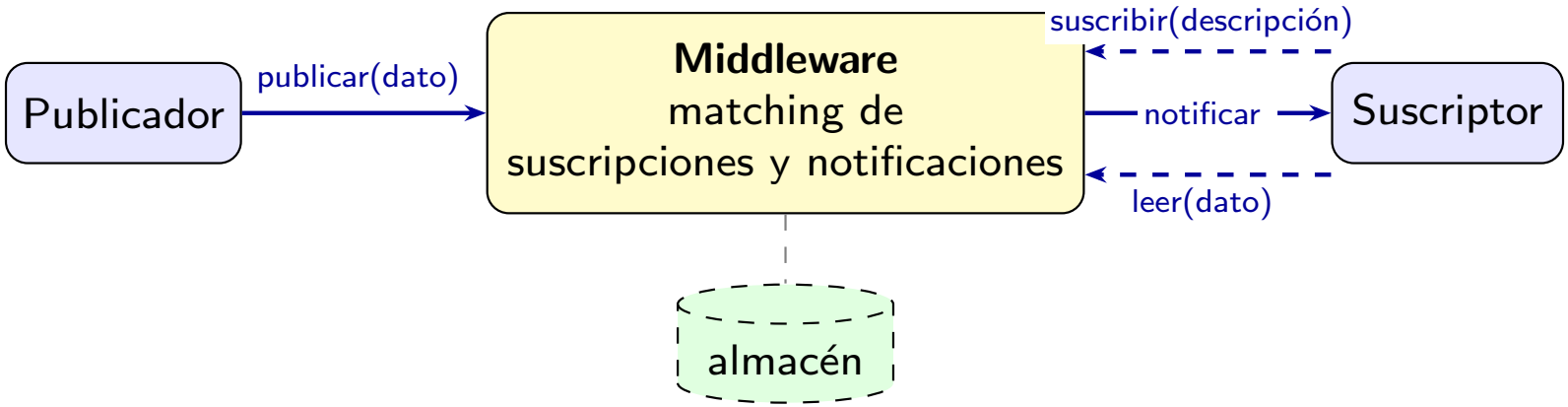
Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Fig. 2.10.

Suscripciones: topic-based y content-based

- **Topic-based**: pares (atributo, valor).
  Ej.: (tema, alertas).
- **Content-based**: pares (atributo, rango).
  Ej.: (temperatura, $[35, \infty)$).

Cuanto más expresiva la suscripción, más caro el *matching*.

¿Se envía el dato o sólo se avisa?



1. **Notificación con el dato**: el middleware no guarda nada.
   $\Rightarrow$ temporalmente acoplado.
2. **Sólo aviso**, y el suscriptor lee después: el middleware guarda el dato (con un *lease*).
   $\Rightarrow$ desacopla también en el tiempo.

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Fig. 2.12.

La matriz para los ejercicios

	Temp. acoplado	**Temp. desacoplado**
Ref. acoplado	**Directa** RPC, HTTP request-reply	**Mailbox** mail, cola de un receptor
Ref. desacoplado	**Event-based** pub/sub que entrega el dato	**Shared data space** tuple space, cola compartida de trabajos, pub/sub que almacena

Para cada conexión entre componentes:

- ¿El emisor tiene que saber quién es el receptor?
- ¿El receptor tiene que estar vivo en ese momento?

Adaptado de M. van Steen & A. S. Tanenbaum, *Distributed Systems*, 4ta ed., Fig. 2.9.

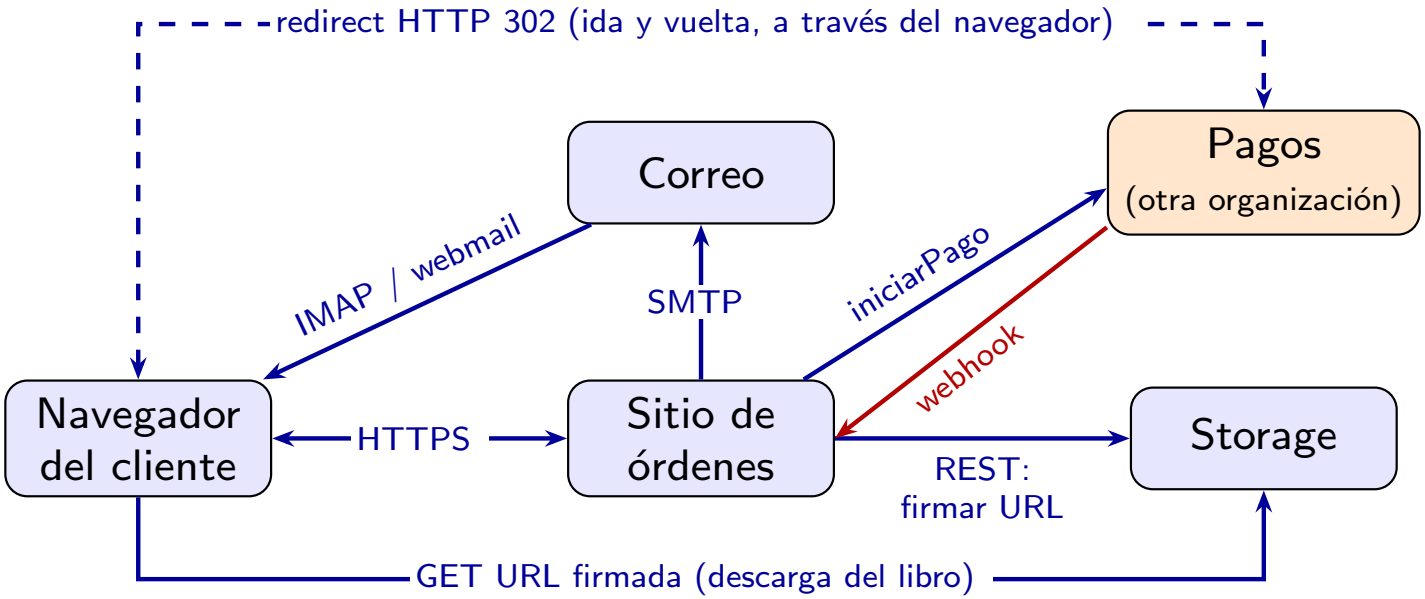
Ejercicio 2:

Considerar un escenario en que se realiza una compra virtual de un libro en formato electrónico con las siguientes componentes:

a. El sitio web para realizar búsquedas y procesar los pedidos de compras.
b. El servidor de storage con los libros en formato digital.
c. El sistema de correo para realizar el delivery de la URL de descarga.
d. El sistema para realizar cobros que redirige al cliente luego de que el pago es realizado y avisa al web server que el pago fue realizado.

Diseñe el sistema distribuido definiendo las interfaces de cada componente y los niveles de procesamiento y datos de cada componente. Analizar qué tipos de acoplamientos referencial y temporal hay.

Ejercicio 2: componentes y conectores



- El libro no pasa por el sitio: el navegador lo baja del storage con una URL firmada que vence.
- El pago va y vuelve **a través del navegador**; la confirmación confiable llega por el webhook.

Ejercicio 2: niveles dentro de cada componente

Componente	**Interfaz**	**Procesamiento**	**Datos**
Sitio de órdenes			
Storage			
Correo			
Pagos			

Ejercicio 2: niveles dentro de cada componente

Componente	**Interfaz**	**Procesamiento**	**Datos**

Sitio de órdenes	`buscar(consulta)`
	`agregarCarrito(libro, usuario)`
	`generarPedido(carrito, usuario)`
	`confirmarPedido(pedido, usuario)`
		búsqueda, carrito, pedidos	catálogo y pedidos

Storage			
Correo			
Pagos			

Ejercicio 2: niveles dentro de cada componente

Componente	**Interfaz**	**Procesamiento**	**Datos**

Sitio de órdenes	`buscar(consulta)`
	`agregarCarrito(libro, usuario)`
	`generarPedido(carrito, usuario)`
	`confirmarPedido(pedido, usuario)`
		búsqueda, carrito, pedidos	catálogo y pedidos

Storage	`obtenerURLFirmada(libro, usuario)` `descargarLibro(url)`	control de acceso, firma de URLs	archivos de los libros
Correo			
Pagos			

Ejercicio 2: niveles dentro de cada componente

Componente	**Interfaz**	**Procesamiento**	**Datos**

Sitio de órdenes	`buscar(consulta)`
	`agregarCarrito(libro, usuario)`
	`generarPedido(carrito, usuario)`
	`confirmarPedido(pedido, usuario)`
		búsqueda, carrito, pedidos	catálogo y pedidos

Storage	`obtenerURLFirmada(libro, usuario)` `descargarLibro(url)`	control de acceso, firma de URLs	archivos de los libros
Correo	`enviar(destino, url)`	colas, reintentos, ruteo	casillas
Pagos			

Ejercicio 2: niveles dentro de cada componente

Componente	**Interfaz**	**Procesamiento**	**Datos**

Sitio de órdenes	`buscar(consulta)`
	`agregarCarrito(libro, usuario)`
	`generarPedido(carrito, usuario)`
	`confirmarPedido(pedido, usuario)`
		búsqueda, carrito, pedidos	catálogo y pedidos

Storage	`obtenerURLFirmada(libro, usuario)` `descargarLibro(url)`	control de acceso, firma de URLs	archivos de los libros
Correo	`enviar(destino, url)`	colas, reintentos, ruteo	casillas
Pagos	`iniciarPago(monto, urlRetorno)`	autorización, antifraude	transacciones

Ejercicio 2: cada conexión en la matriz

Conexión	**Referencial**	**Temporal**	**Tipo**
Navegador → storage			
Sitio → correo			
Correo → cliente			
Sitio → pagos (redirect)			
Pagos → sitio (webhook)			

Ejercicio 2: cada conexión en la matriz

Conexión	**Referencial**	**Temporal**	**Tipo**
Navegador → storage	Acoplado: conoce la URI	Acoplado: request-reply	Directa
Sitio → correo			
Correo → cliente			
Sitio → pagos (redirect)			
Pagos → sitio (webhook)			

Ejercicio 2: cada conexión en la matriz

Conexión	**Referencial**	**Temporal**	**Tipo**
Navegador → storage	Acoplado: conoce la URI	Acoplado: request-reply	Directa
Sitio → correo	Acoplado: servidor y casilla destino	Desacoplado: store-and-forward	Mailbox
Correo → cliente			
Sitio → pagos (redirect)			
Pagos → sitio (webhook)			

Ejercicio 2: cada conexión en la matriz

Conexión	**Referencial**	**Temporal**	**Tipo**
Navegador → storage	Acoplado: conoce la URI	Acoplado: request-reply	Directa
Sitio → correo	Acoplado: servidor y casilla destino	Desacoplado: store-and-forward	Mailbox
Correo → cliente	Acoplado: su casilla	Desacoplado	Mailbox
Sitio → pagos (redirect)			
Pagos → sitio (webhook)			

Ejercicio 2: cada conexión en la matriz

Conexión	**Referencial**	**Temporal**	**Tipo**
Navegador → storage	Acoplado: conoce la URI	Acoplado: request-reply	Directa
Sitio → correo	Acoplado: servidor y casilla destino	Desacoplado: store-and-forward	Mailbox
Correo → cliente	Acoplado: su casilla	Desacoplado	Mailbox
Sitio → pagos (redirect)	Acoplado	Acoplado	Directa
Pagos → sitio (webhook)			

Ejercicio 2: cada conexión en la matriz

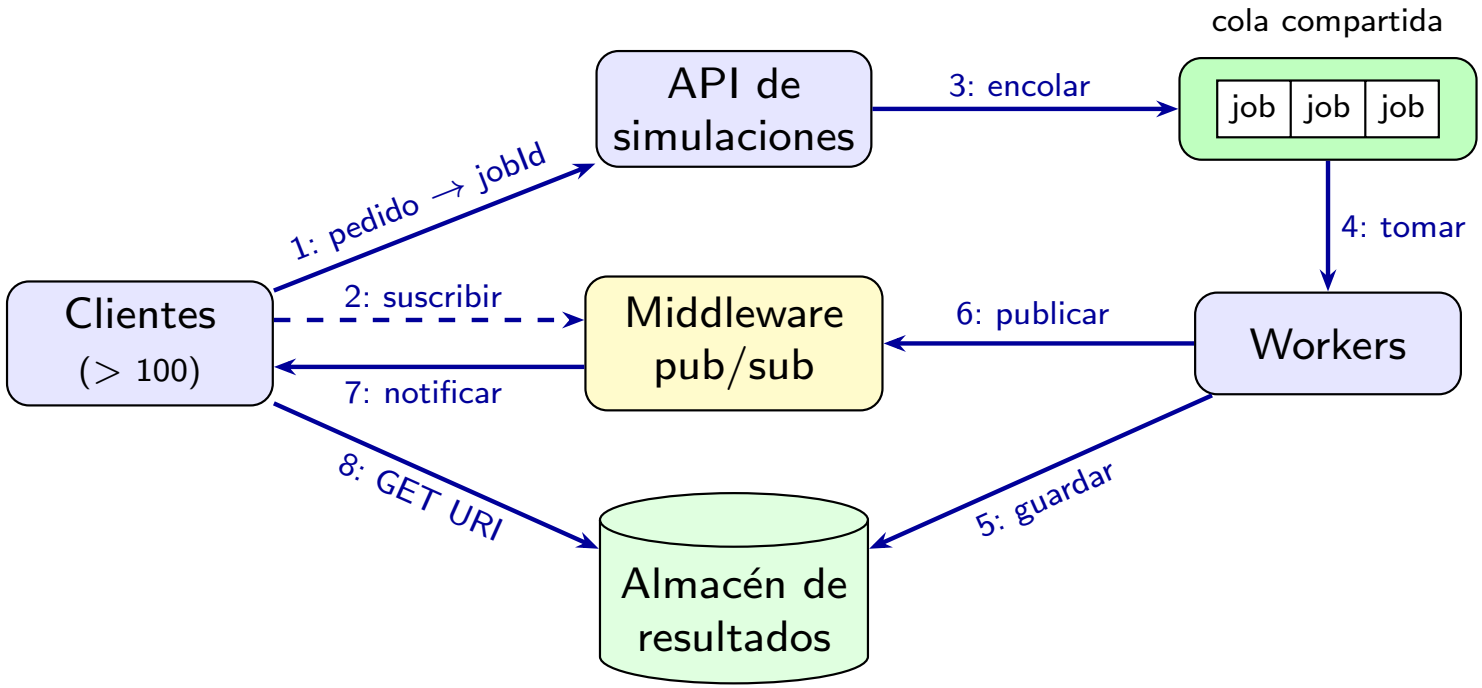
Conexión	**Referencial**	**Temporal**	**Tipo**
Navegador → storage	Acoplado: conoce la URI	Acoplado: request-reply	Directa
Sitio → correo	Acoplado: servidor y casilla destino	Desacoplado: store-and-forward	Mailbox
Correo → cliente	Acoplado: su casilla	Desacoplado	Mailbox
Sitio → pagos (redirect)	Acoplado	Acoplado	Directa
Pagos → sitio (webhook)	Acoplado: URL de notificación	Acoplado	Directa

Ejercicio 3:

Considere un servicio de simulaciones de eventos atmosféricos en el que la ejecución de cada modelo puede tardar entre 10-15 minutos, dependiendo de parámetros de ejecución que se definen con cada pedido de ejecución.

1. Diagramar las componentes de un sistema distribuido usando la técnica de publicaciones y suscripciones.
2. Mencione ventajas y desventajas de usar un sistema basado en eventos (event-based) frente a un sistema de almacenamiento compartido (shared-space).
3. ¿Cómo resolvería el esquema de notificaciones? ¿Enviaría el resultado de las simulaciones con la notificación o sólo anunciaría para que el cliente lo pida?

Ejercicio 3: diagrama



- El cliente pide la simulación, recibe un `jobId` y se suscribe a `job.⟨jobId⟩.done`.
- Un worker toma el trabajo de la cola, corre 10–15 minutos, guarda el resultado y publica.

Ejercicio 3: event-based vs. shared data space

	Event-based	**Shared data space**
Acoplamiento	Ref. desacoplado, temp. acoplado	Desacoplado en ambos ejes
Cliente desconectado cuando termina el job	Pierde la notificación	Lee el resultado cuando vuelve
Almacenamiento en el middleware	No	Sí, con leases
Latencia de entrega	Menor	Mayor: hay una lectura posterior

Ejercicio 3: event-based vs. shared data space

	Event-based	**Shared data space**
Acoplamiento	Ref. desacoplado, temp. acoplado	Desacoplado en ambos ejes
Cliente desconectado cuando termina el job	Pierde la notificación	Lee el resultado cuando vuelve
Almacenamiento en el middleware	No	Sí, con leases
Latencia de entrega	Menor	Mayor: hay una lectura posterior

Con trabajos de 15 minutos y más de 100 clientes manda el **desacople temporal**: shared data space, o pub/sub con suscripciones persistentes.

Ejercicio 3: notificaciones

¿Enviar el resultado con la notificación o sólo anunciarlo?

Ejercicio 3: notificaciones

¿Enviar el resultado con la notificación o sólo anunciarlo?

Notificar con la URI del resultado, sin adjuntarlo

- Los resultados de una simulación son grandes.
- La descarga se puede reanudar si se corta.
- La retención se maneja con leases.

Ejercicio 3: ¿cuántos workers?

Si sobra tiempo

Llega una ráfaga de 100 pedidos y hay W workers. La última tanda termina a

$$\left\lceil \frac{100}{W} \right\rceil \times 15 \text{ min.}$$

Para que nadie espere más de 30 minutos:
$\lceil 100/W \rceil \leq 2 \Rightarrow W \geq 50$.

¿Preguntas?

